P2: Looking for group synchronization findings

Deadlock Risk - Circular Wait

- Occurs when threads hold resources while indefinitely waiting on each other.

Example: Thread A holds a Tank and waits for a Healer AND Thread B holds a Healer token and waits for a Tank.

Result: System freezes permanently in a **circular dependency loop**

Starvation Risk - Resource Deprivation

- Occurs when threads are **indefinitely delayed even with the system making progress**

Role Shortages: DPS queue drains 3x faster (1 Tank : 1 Healer : 3 DPS). Partial parties hold resources while waiting for missing roles.

Slot Competition: Threads may get repeatedly bypassed by faster CPU scheduling when trying to claim available dungeon instances

Synchronization Mechanisms to Avoid Deadlock

1. Strict Canonical Resource Ordering
   - I threads acquire role semaphores in the exact same sequence:

Tank -> Healer-> DPS 1-> DPS 2-> DPS 3 -> Dungeon Slot

```
sem_wait(&tank_queue);
sem_wait(&healer_queue);
sem_wait(&dps_queue);
sem_wait(&dps_queue);
sem_wait(&dps_queue);
```

Circular wait is impossible because no thread ever requests a Tank while holding a Healer

Synchronization Mechanisms to Avoid Deadlock

2. Pre-Calculated Capacity Bound

- engine() calculates total valid parties before spawning threads:

```
int dps_parties = d / 3; // each party requires 3 DPS players
int max_parties = t;
if (h < max_parties) max_parties = h;
if (dps_parties < max_parties) max_parties = dps_parties;
```

This makes sure every spawned thread has enough roles in the semaphores to finish execution

Synchronization Mechanisms to Avoid Deadlock

3. Non-Overlapping Lock Scoping

- Role semaphores assemble parties before requesting instance slots

```
pthread_mutex_lock(&instance_mutex);
for (int i = 0; i < global_n; i++) {
    if (!instances[i].is_active)
    {
        instances[i].is_active = true;
        assigned_instance = i;
        break;
    }
```

```
pthread_mutex_unlock(&instance_mutex);
```

instance_mutex is held only briefly for atomic slot lookups to avoid nested lock deadlocks

Synchronization Mechanisms to Avoid Starvation

1. POSIX Semaphore FIFO Scheduling
- OS kernel maintains FIFO ordering for threads blocked at sem_wait().

```
sem_wait(&dungeon_instances);
```

```
sleep(clear_time);
```

```
sem_post(&dungeon_instances);
```

Threads that wait longer are guaranteed priority when sem_post() frees a resource to prevent line jumping

Synchronization Mechanisms to Avoid Starvation

2. Bounded Resource Lifespans

- Every thread in a dungeon releases its slot after a finite time:

```
sleep(clear_time);

// leave and mark as empty
pthread_mutex_lock(&instance_mutex);
instances[assigned_instance].is_active = false;
printf("\n <- [Party %d] CLEARED Instance %d\n", party_id, assigned_instance + 1);
print_instance_status(); //print status
pthread_mutex_unlock(&instance_mutex);

//release instance slot
sem_post(&dungeon_instances);
```

Continuous recycling guarantees waiting parties will always gain entry

Synchronization Mechanisms to Avoid Starvation

3. Fine-Grained Critical Sections

- Mutexes protect small immediate operations (updating state arrays, printf output).

```
pthread_mutex_lock(&log_mutex);
printf("[Party %d] Formed! Waiting for an available dungeon instance...\n", party_id);
pthread_mutex_unlock(&log_mutex);
```

Minimizes lock contention so worker threads spend near-zero time waiting on mutual exclusion locks